

Natural Language Processing

10. Masked Language Models

Paolo Torrioni

Fall 2025

Notice

Credits

The present slides are largely an adaptation of existing material, including slides by [Jurafsky & Martin](#), [Manning](#), [Potts & MacCartney](#), [Black & Mortensen](#), and others, to whom I am very grateful.

Downloading and sharing

A copy of these slides can be downloaded from [virtuale](#) and stored for personal use only. Please do not redistribute.

Table of Contents

- Bidirectional Transformer Encoders
- Training Bidirectional Encoders
- Contextual Embeddings
- Fine-Tuning for Classification
- Fine-Tuning for Sequence Labeling

Bidirectional Transformer Encoders

Bidirectional Encoders

- The generative transformer architectures for **language modeling** seen so far rely on **causal attention**
- However, there are tasks, other than language modeling, can benefit from **bi-directional context** (seen with bi-RNNs)
- Architectures that consider the context from both sides are **encoders** capable of producing **contextual embeddings**: representations of words in context
- Such encoders can be used
 - as general-purpose **feature extractors**, feeding subsequent stages of an NLP pipeline with high-quality input representations, or
 - as pre-trained deep models that can be **adapted** by fine-tuning to perform well in specific tasks

Three Architectures for Language Models

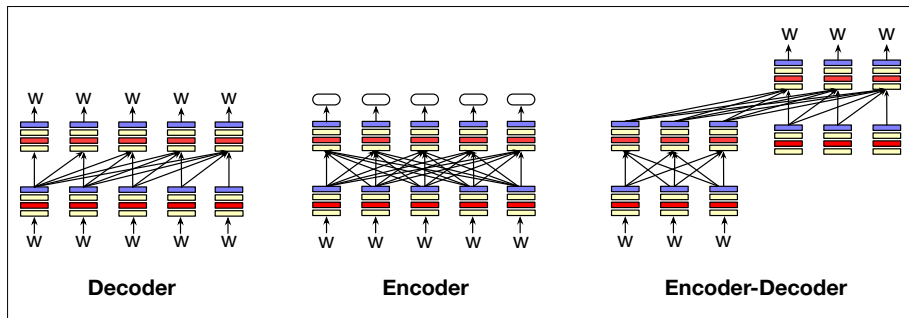
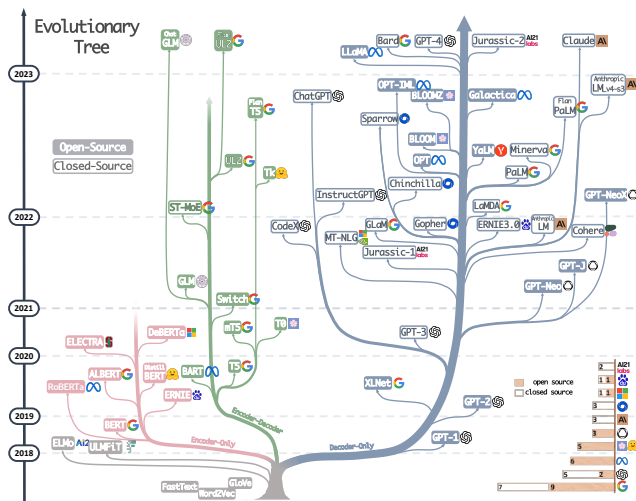


Figure 7.3 Three architectures for language models: decoders, encoders, and encoder-decoders. The arrows sketch out the information flow in the three architectures. Decoders take tokens as input and generate tokens as output. Encoders take tokens as input and produce an encoding (a vector representation of each token) as output. Encoder-decoders take tokens as input and generate a series of tokens as output.

- The first widely-used transformer-based encoder model was **BERT**

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding, Devlin et al, 2018

Evolution of Transformer Models



Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond, Yang et al, 2023

Bidirectional Transformer Encoders

- The focus of **encoder-only** models is computing contextualized representations
- Tasks where bi-directional context is useful include interpretative tasks such as **text classification** and **sequence labeling**
 - part-of-speech tagging, parsing, named entity recognition
- Self-attention is used to map sequences of input embeddings

$$x_1, x_2, \dots, x_n$$

to same-length sequences of output embeddings

$$h_1, h_2, \dots, h_n$$

where the output vectors have been contextualized based on the entire input sequence

Architecture of encoder models

Two main differences from decoder models:

- Attention is not causal
- Training is different

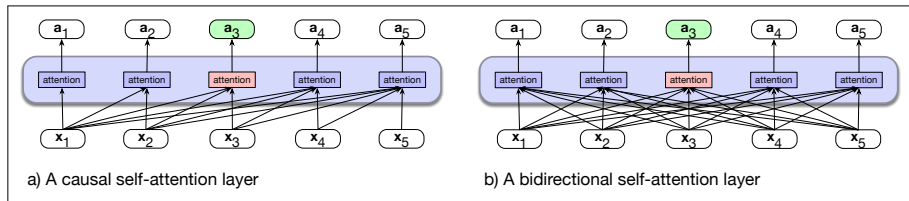


Figure 11.1 (a) The causal transformer from Chapter 9, highlighting the attention computation at token 3. The attention value at each token is computed using only information seen earlier in the context. (b) Information flow in a bidirectional attention model. In processing each token, the model attends to all inputs, both before and after the current one. So attention for token 3 can draw on information from following tokens.

Attention Mask

- The way to implement the two types of attention is by using an **attention mask**, i.e., a matrix **M** in which

$$M_{ij} = -\infty \forall j > i$$

which is *added* to the $\mathbf{QK}^T$ matrix

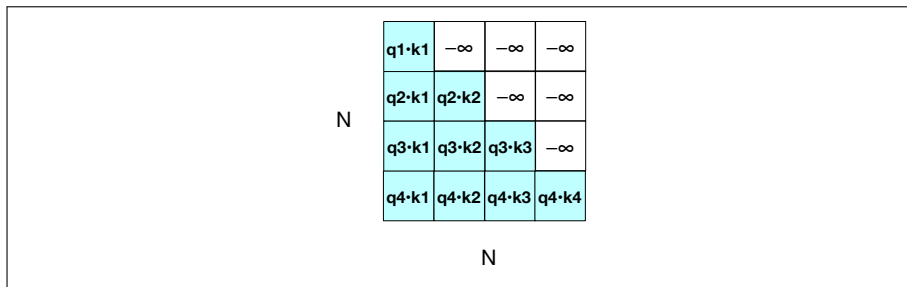


Figure 9.9 The $N \times N$ $\mathbf{QK}^T$ matrix showing the $q_i \cdot k_j$ values, with the upper-triangle portion of the comparisons matrix zeroed out (set to $-\infty$, which the softmax will turn to zero).

Attention Mask

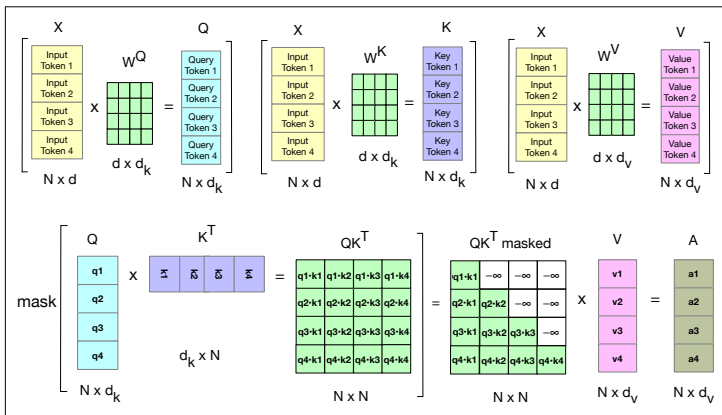


Figure 9.10 Schematic of the attention computation for a single attention head in parallel. The first row shows the computation of the Q , K , and V matrices. The second row shows the computation of QK^T , the masking (the softmax computation and the normalizing by dimensionality are not shown) and then the weighted sum of the value vectors to get the final attention vectors.

Attention Mask

- Hence, to implement bi-directional attention, one simply doesn't apply the attention mask

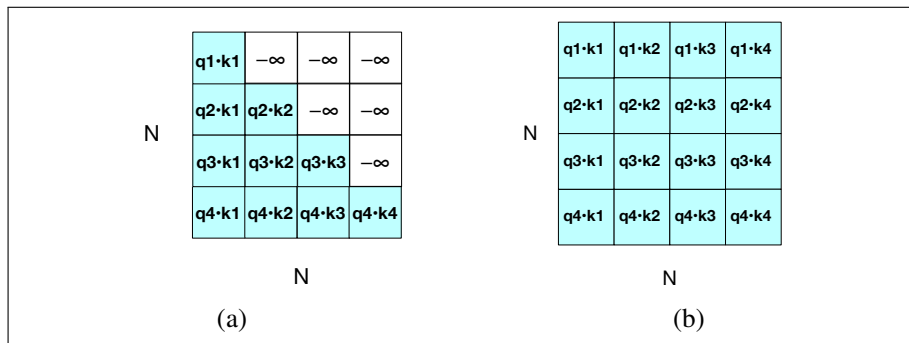


Figure 11.2 The $N \times N$ $\mathbf{QK}^T$ matrix showing the $q_i \cdot k_j$ values, with the upper-triangle portion of the comparisons matrix zeroed out (set to $-\infty$, which the softmax will turn to zero).

Examples

- BERT
 - English-only subword vocabulary, $|\mathcal{V}| = 30,000$, generated with WordPiece
 - Hidden layers with $d = 768$
 - 12 layers of transformer blocks, with 12 heads
 - Input context window 512 tokens
 - Overall, about 100M parameters
- XLM-RoBERTa, trained on 100 languages
 - Multilingual subword vocabulary, $|\mathcal{V}| = 250,000$, generated with SentencePiece Unigram LM
 - Hidden layers with $d = 1024$
 - 24 layers of transformer blocks, with 16 heads
 - Input context 512 tokens
 - Overall, about 550M parameters
- Notice: MLM usually $\ll$ causal LM (Llama3 has 400B+ parameters)

Training Bidirectional Encoders

Training Encoders

- The task used to train encoder models is a **cloze task** (fill-in-the-blank)
- It's a form of **denoising**
- Instead of predicting the next word, we're asked to predict a missing item given the rest of the sentence

The [MASK] of Walder Pond is so beautifully ...

- During training, the input is deprived of one or more elements
- The model generates a probability distribution over $\mathcal{V}$ for each missing element
- Learning is driven by **cross-entropy loss from each prediction**
- Other possible manipulations besides masking: substitutions, reorderings, deletions, extraneous insertions, etc.

Masked Language Modeling

- BERT defined the **Masked Language Modeling** task
- Uses unannotated text from a large corpus
- A random sample of tokens from each training sequence (15%) is selected for use in the learning task
- Chosen tokens may be:
 - Either replaced with the [MASK] special token (80%)
 - Or, replaced with a different token from $\mathcal{V}$ (10%)
 - Or, left unchanged (10%)
- Notice: somewhat inefficient, since only 15% of input is used for training weights

Masked Language Modeling

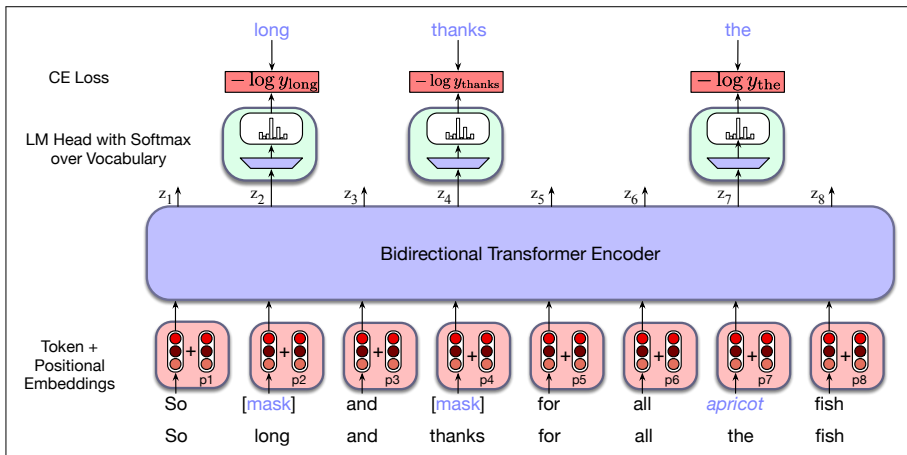


Figure 11.3 Masked language model training. In this example, three of the input tokens are selected, two of which are masked and the third is replaced with an unrelated word. The probabilities assigned by the model to these three items are used as the training loss. The other 5 tokens don't play a role in training loss.

Training Regimes

- BERT was training on 3.3B words (English Wikipedia and BooksCorpus)
- Modern MLMs are now trained on much larger datasets of web data, filtered, and augmented with higher-quality data like Wikipedia
- Multilingual models use web data and multilingual Wikipedia
 - For example, XLM-R on 300B tokens in 100 languages via Common Crawl
 - A problem is what data to use to build $\mathcal{V}$: what text should be used to learn multilingual tokenization, given that languages are not uniformly represented in the Web?
 - One possibility is to up-weight the probability of less-represented languages
 - Why does this matter?

Tokenizer Fertility

- The tokenizer fertility measures the average amount of tokens produced per tokenized word
- The tokenizer fertility is strictly correlated with the inference speed of the model with respect to a specific language (longer sequences take more time and resources to process/generate)

Fertility computed over a sample of Cultura X (CX) data and Wikipedia (Wp):

Model	Voc. Size	Fertility IT (CX)	Fertility EN (CX)	Fertility IT (Wp)	Fertility EN (Wp)
Mistral-7B-v0.1	32000	1.87	1.32	2.05	1.57
gemma-7b	256000	1.42	1.18	1.56	1.34
Minerva-3B-base-v1.0	32768	1.39	1.32	1.66	1.59

From <https://huggingface.co/sapienzanlp/Minerva-3B-base-v1.0>

Monolingual or Multilingual?

- Multilingual models can be more practical than several monolingual models
- Advantage for **low-resource languages**: can leverage linguistic information from similar languages
- However, **curse of multilinguality**: the performance on each language degrades
- Also, grammatical structures in higher-resource languages bleed into lower resource languages

Contextual Embeddings

Contextual Embeddings

- Static embeddings represent the meaning of *word types*
- Contextual embeddings represent the meaning of *word instances*

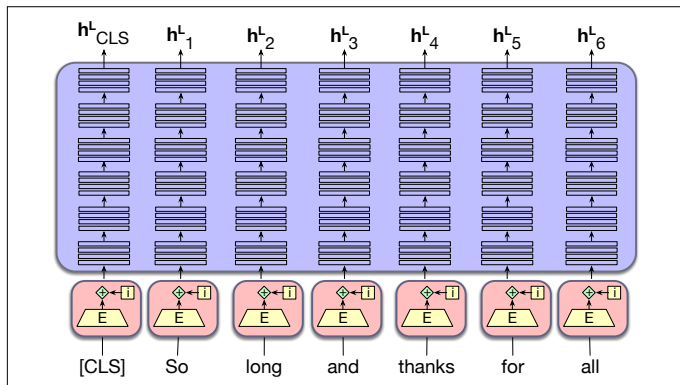


Figure 11.5 The output of a BERT-style model is a contextual embedding vector h_i^L for each input token x_i .

Word Sense Disambiguation

- Important for **Natural Language Understanding** tasks like **Word Sense Disambiguation (WSD)**

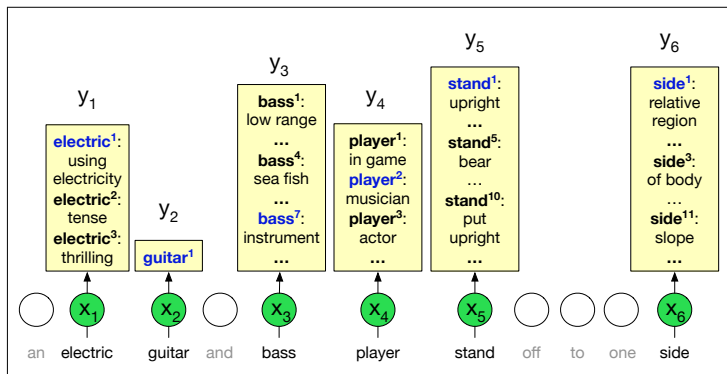


Figure 11.7 The all-words WSD task, mapping from input words (x) to WordNet senses (y). Figure inspired by [Chaplot and Salakhutdinov \(2018\)](#).

Word Sense Disambiguation

- The fact that context disambiguates senses can also be visualized geometrically

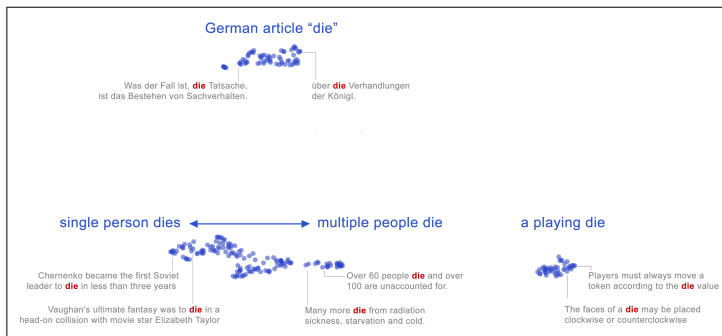


Figure 11.6 Each blue dot shows a BERT contextual embedding for the word *die* from different sentences in English and German, projected into two dimensions with the UMAP algorithm. The German and English meanings and the different English senses fall into different clusters. Some sample points are shown with the contextual sentence they came from. Figure from [Coenen et al. \(2019\)](#).

Word Sense Disambiguation

Possible approach:

- Use BERT as encoder; for example, for each input token x_i take the sum of the last 4 layers' vector representations to obtain $\mathbf{v}_i$
- Use a sense-labeled training corpus ([here](#)) like SemCor OMSTI or SenseEval
- For each sense s of any word in the corpus, for each of the n tokens of that sense, compute a **sense embedding**

$$\mathbf{v}_s = \frac{1}{n} \sum_i \mathbf{v}_i$$

- Predict $\text{sense}(t) = \operatorname{argmax}_{s \in \text{senses}(t)} \text{cosine}(\mathbf{t}, \mathbf{v}_s)$

Word Sense Disambiguation

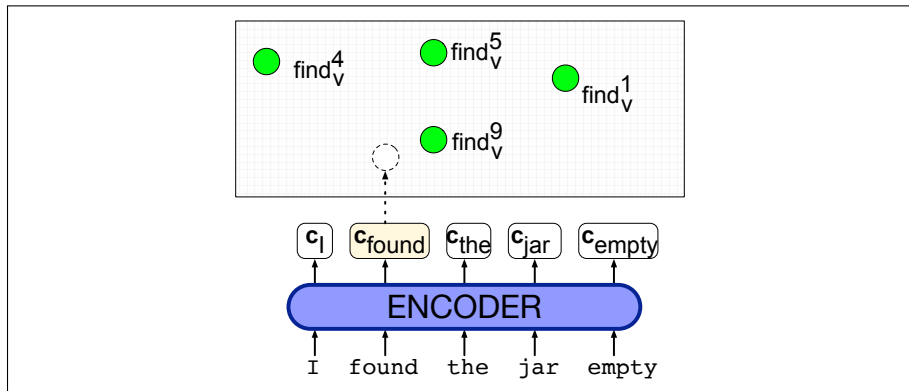


Figure 11.8 The nearest-neighbor algorithm for WSD. In green are the contextual embeddings precomputed for each sense of each word; here we just show a few of the senses for *find*. A contextual embedding is computed for the target word *found*, and then the nearest neighbor sense (in this case **find_v⁹**) is chosen. Figure inspired by Loureiro and Jorge (2019).

Fine-Tuning for Classification

Text Classification

- The task is to classify an entire sequence (of text) with a single label
- Sequence encoding can be done in many ways (seen previously)
- With encoders like BERT, we have a special dedicated input token [CLS]
- The output vector in the final layer of the [CLS] token can be fed to a **classifier head** (e.g. LR or FFN) to be trained accordingly

$$\mathbf{y} = \text{softmax}(\mathbf{h}_{CLS}^L \mathbf{W}_C)$$

- Fine-tuning $\mathbf{W}_C$ requires supervised training data
- As usual, cross-entropy loss drives learning
- Some layers of the encoder can be left *trainable* (as opposed to all *frozen*) for possible further improvement
- *Notice*: each time we fine-tune a model, we trade-off with the model's ability to process new out-of-domain data (**generalization** capability)

Example: Sentiment Classification

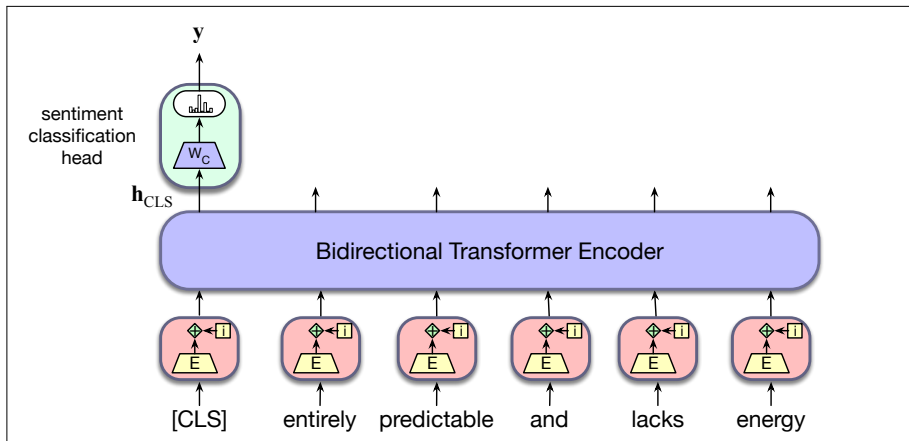


Figure 11.9 Sequence classification with a bidirectional transformer encoder. The output vector for the `[CLS]` token serves as input to a simple classifier.

Sequence-Pair Classification

- Examples: paraphrase detection, natural language inference (recognizing textual entailment), discourse coherence
- To fine-tune for these tasks, input sequence pairs are separated with a special [SEP] token
- The classification is made using the [CLS] token's output vector, as usual

Fine-Tuning for Sequence Labeling

Named Entity Recognition

- A typical sequence labeling task is **Named Entity Recognition**
- **Named entity**: many things (depends on application)
 - anything that can be referred to by proper name
 - people
 - places
 - organizations
 - gene or protein names
 - financial assets
 - vehicles
 - etc.
 - temporal expressions
 - numerical expressions (prices)

Type	Tag	Sample Categories	Example sentences
People	PER	people, characters	Turing is a giant of computer science.
Organization	ORG	companies, sports teams	The IPCC warned about the cyclone.
Location	LOC	regions, mountains, seas	Mt. Sanitas is in Sunshine Canyon .
Geo-Political Entity	GPE	countries, states	Palo Alto is raising the fees for parking.

Figure 11.10 A list of generic named entity types with the kinds of entities they refer to.

Named Entity Recognition

- **NER:** finding spans of text that constitute proper names and then classifying the type of the entity
 - Segmentation ambiguities
 - Type ambiguities

[PER Washington] was born into slavery on the farm of James Burroughs.
[ORG Washington] went up 2 games to 1 in the four-game series.
Blair arrived in [LOC Washington] for what may well be his last state visit.
In June, [GPE Washington] passed a primary seatbelt law.

Figure 11.11 Examples of type ambiguities in the use of the name *Washington*.

Sequence Labeling Tagging Schemes

- Tagging schemes like **BIO**, **IO**, or **BIOES** allow us to treat NER like a word-by-word sequence labeling task

Words	IO Label	BIO Label	BIOES Label
Jane	I-PER	B-PER	B-PER
Villanueva	I-PER	I-PER	E-PER
of	O	O	O
United	I-ORG	B-ORG	B-ORG
Airlines	I-ORG	I-ORG	I-ORG
Holding	I-ORG	I-ORG	E-ORG
discussed	O	O	O
the	O	O	O
Chicago	I-LOC	B-LOC	S-LOC
route	O	O	O
.	O	O	O

Figure 11.12 NER as a sequence model, showing IO, BIO, and BIOES taggings.

Fine-Tuning for Sequence Labeling

- We pass the final output vector of each input token to a classifier
- With k classes, the set of weights to be learned is $\mathbf{W}_K \in \mathbb{R}^{d \times k}$
- The output of the NER head can be passed to a **CRF layer** to take global tag-level transitions into account

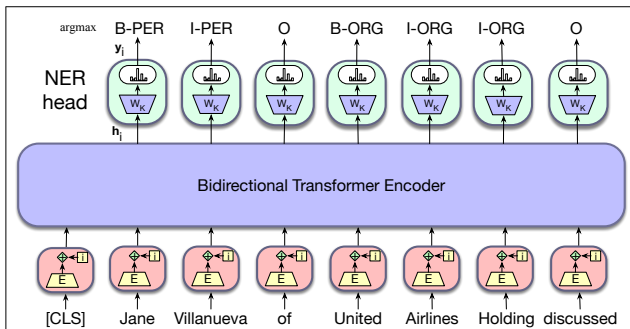


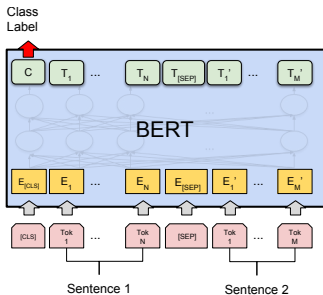
Figure 11.13 Sequence labeling for named entity recognition with a bidirectional transformer encoder. The output vector for each input token is passed to a simple k -way classifier.

Evaluation of NER

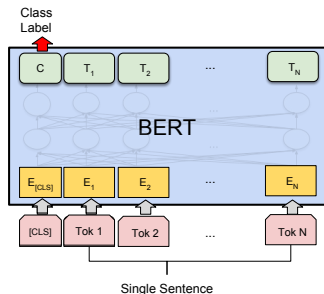
- Typically, **recall**, **precision**, and **F₁**
 - Recall: $\frac{\text{number of correctly labeled responses}}{\text{total that should have been labeled}}$
 - Precision: $\frac{\text{number of correctly labeled responses}}{\text{total that has been labeled}}$
 - F₁: harmonic mean of the two
 - The entity is the unit of response
 - *Jane Villanueva* is a single response
 - Effect of segmentation: a system that labels *Jane* but not *Jane Villanueva* as PER causes two errors (a false positive for O and a false negative for I-PER)

Summary of Fine-Tuning Procedure

- Typically very fast (compare to pre-training)
- Most model hyperparameters are the same as in pre-training
- Evaluation against benchmarks like GLUE
 - Sentence Pair & Single Sentence Classification, QA, Tagging



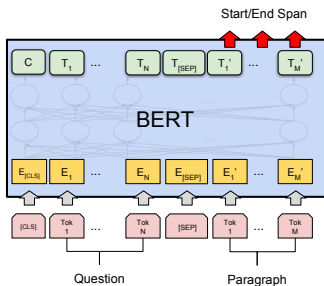
(a) Sentence Pair Classification Tasks:
MNLI, QQP, QNLI, STS-B, MRPC,
RTE, SWAG



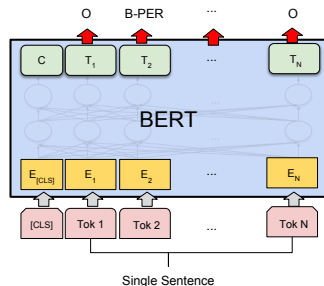
(b) Single Sentence Classification Tasks:
SST-2, CoLA

Summary of Fine-Tuning Procedure

- Typically very fast (compare to pre-training)
- Most model hyperparameters are the same as in pre-training
- Evaluation against benchmarks like GLUE
 - Sentence Pair & Single Sentence Classification, QA, Tagging



(c) Question Answering Tasks:
SQuAD v1.1



(d) Single Sentence Tagging Tasks:
CoNLL-2003 NER

Evaluating Masked Language Models

- **GLUE**: General Language Understanding Evaluation benchmark
 - Evaluate NLU model ability to process language in a way that is not exclusive to a single task, genre, or dataset
 - Tasks with limited training data to encourage models that share general linguistic knowledge across tasks
- Tasks: **single-sentence**, **similarity**, paraphrase, **inference**

GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding, A Wang et al, ICLR 2019
SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems, A Wang et al, NeurIPS 2019
Beyond the Imitation Game: Quantifying and extrapolating the capabilities of language models, A Srivastava et al, 2022

Evaluating Masked Language Models

- **GLUE**: General Language Understanding Evaluation benchmark
- Single-sentence tasks
 - **CoLA**: English grammaticality of sentence
 - **SST-2**: Stanford Sentiment Treebank
- Similarity and paraphrase tasks
 - **MRPC**: semantic equivalence of sentences from news sources
 - **QQP**: semantic equivalence of Quora question pairs
 - **STS-B**: semantic equivalence of news headlines
- Inference tasks
 - **MLNI**: does the premise entail the hypothesis?
 - **QNLI**: Stanford Question Answering dataset
 - **RTE**: Recognizing Textual Entailment
 - **WNLI**: Winograd Schema Challenge

Questions?